

Reto 3 — Resumen de Entrega

Reto 3 — Automatización Mobile

Máster Profesional en QA y Automatización de Pruebas

Autora: Adriana Troche **Fecha de entrega:** 25 de mayo de 2026

1. Aplicación bajo prueba

- **App:** My Demo App (SauceLabs)
- **Android:** `com.saucelabs.mydemoapp.android` (v2.2.0)
- **iOS:** SauceLabs Demo App Simulator (v2.2.2)

2. Stack técnico

Componente	Tecnología
Lenguaje	TypeScript
Test runner	WebdriverIO v9
Framework	Mocha (BDD)
Motor de automatización	Appium 2
Driver Android	UiAutomator2
Driver iOS	XCUITest
Cloud iOS	SauceLabs (free trial)
Reportes	Allure
CI/CD	Jenkinsfile declarativo
Patrón	Page Object Model (POM)

3. Escenarios automatizados (funcionalidades diferentes por SO)

Android — Flujo crítico de COMPRA E2E (`purchase-flow.e2e.ts`)

- 1. Catálogo → menú → Login (`bod@example.com` / `10203040`)
- 2. Selección de producto Sauce Labs Backpack
- 3. Add to Cart + validación
- 4. Checkout: shipping address (7 campos)
- 5. Checkout: payment info (4 campos)
- 6. Review Order + Place Order
- 7. Validación de confirmación "Checkout Complete"

11 pasos verificados | Tiempo: 34.6s | Resultado: PASS 

iOS — Catálogo: navegación + detalle (`catalog-navigation.e2e.ts`)

- 1. Navegación entre tabs (Cart → Catalog)
- 2. Validación de 6 productos visibles
- 3. Tap en primer producto
- 4. Validación de precio, descripción, botón Add to Cart
- 5. Swipe horizontal en galería de imágenes (gesto nativo W3C Actions)
- 6. Calificación con 5 estrellas

7 pasos verificados | Tiempo: 42.2s | Resultado: PASS 

4. Justificación de viabilidad de automatización

Cada escenario fue evaluado contra **6 criterios**:

Criterio	Compra Android	Catálogo iOS
Estabilidad del flujo	Alta	Alta
ROI (frecuencia ejecución)	Muy alto (cada release)	Alto (regresión visual)
Criticidad de negocio	Crítica (sin checkout no hay revenue)	Media (descubrimiento)
Repetibilidad determinista	Sí (datos hardcoded)	Sí (sort es función pura)
Factibilidad técnica Appium	Sí	Sí
Independencia de datos	Sí	Sí

→ Ambos cumplen los 6 criterios → automatización justificada.

5. Resultados finales

Métrica	Valor
Tests implementados	2
Tests pasando	2 de 2 (100%)
Pass rate Allure	100%
Page Objects	9 (7 Android + 2 iOS)
Screenshots de evidencia	11 organizadas
Plataformas cubiertas	Android local + iOS SauceLabs cloud

6. Entregables

- ✅ **Repositorio público GitHub:** <https://github.com/adrianagit87/reto3-mobile-automation>
- ✅ **README.md** con instrucciones completas de setup, ejecución y troubleshooting
- ✅ **Jenkinsfile** declarativo con 7 stages parametrizables (RUN_ANDROID, RUN_IOS)
- ✅ **EVIDENCIAS.md** (también en PDF) con capturas del flujo Android, del reporte Allure y métricas
- ✅ **Run iOS en SauceLabs (video MP4 disponible):**
<https://app.saucelabs.com/tests/22c50ba591f74ca4bb89c368f35e2e7c>

7. Arquitectura del proyecto

```

reto3-mobile-automation/
├── README.md, EVIDENCIAS.md, Jenkinsfile
├── config/                3 configs WDI0 (shared + Android + iOS)
├── src/
│   ├── pages/
│   │   ├── BasePage.ts    Clase abstracta del POM
│   │   ├── android/       7 POs (UiAutomator2)
│   │   └── ios/           2 POs (XCUITest)
│   └── helpers/gestures.ts W3C Actions cross-platform
├── test/specs/
│   ├── android/purchase-flow.e2e.ts
│   └── ios/catalog-navigation.e2e.ts
└── docs/evidencias/      11 capturas

```

Principios aplicados

- **POM canónico:** los tests JAMÁS tocan `$()` directamente, todo va por Page Objects

- **POs separados por SO:** locators de UiAutomator2 (Android) y XCUITest (iOS) son distintos
- **BasePage abstracta:** centraliza `waitForLoaded`, `tap`, `setValue`, `readText`
- **Screenshots automáticos en fallo:** hook `afterTest` los adjunta al reporte Allure

8. Comandos clave

```
# Setup inicial
npm install
cp env.example .env # completar credenciales SauceLabs

# Ejecución
npm run test:android    # Android local
npm run test:ios        # iOS SauceLabs cloud
npm run test:all        # Ambos secuencialmente

# Reporte
npm run clean           # IMPORTANTE: limpiar antes de regenerar
npm run report          # Genera + abre Allure HTML
```

Estado de la entrega:  **COMPLETA** — todos los entregables del reto están listos en el repositorio público.